

CALCUL D'UN COUPLAGE DANS UN GRAPHE BIPARTI

Vendredi 12 décembre 2025

AKKAYA Tolunay

tolunay.akkaya@etu.univ-orleans.fr

Université d'Orléans

PAUL Nicolas

nicolas.paul1@etu.univ-orleans.fr

Université d'Orléans

Introduction

Ce rapport présente le détail d'un travail réalisé dans le cadre du cours de Modélisation, Graphes et Algorithmes à l'Université d'Orléans. Le projet cherche à implanter et appliquer l'algorithme de Hopcroft-Karp (Hopcroft, 1973) en OCaml (Leroy, 1990).

Ce document présente donc notre travail d'implantation et nos réponses.

Réponses aux questions

Question 1

Le graphe représentant l'échiquier mutilé est un graphe bi-parti parce qu'il est simple et non-orienté et il admet une 2-coloration. Notre preuve se déroule en trois étapes : construire le graphe, démontrer qu'il admet une 2-coloration, démontrer que tout graphe simple et non-orienté admettant une 2-coloration est bi-parti.

Premièrement, nous cherchons à construire le graphe représentant l'échiquier mutilé. Soit $n \in \mathbb{N}$ et $M = (m_{i,j}) \in \{0;1\}^{n^2}$ la matrice représentant l'échiquier mutilé, où $m_{i,j} = 0$ si la case i,j est mutilée, 1 sinon. Ainsi, posons $V = \{(i,j) \in [1;n]^2 : m_{i,j} = 1\}$ et $E = \{(i,j), (i',j') : (i,j) \in V, (i',j') \in V, |i' - i| + |j' - j| = 1\}$. Donc $G = (V, E)$ est un graphe simple et non-orienté représentant l'échiquier mutilé.

Deuxièmement, nous voulons prouver que notre graphe admet une 2-coloration. Soit $c(i,j) = i + j \bmod 2, i,j \in [1;n]$ une 2-coloration. $\forall (i,j), (i',j') \in E$, et par définition de E , soit $i = i', |j' - j| = 1 \Rightarrow j' = j \pm 1$, soit $j = j', |i' - i| = 1 \Rightarrow i' = i \pm 1$. Alors $i' + j' = i + j \pm 1$. Donc $c(i',j') = i' + j' \bmod 2 = i + j \pm 1 \bmod 2 = i + j \bmod 2 \oplus 1$, $\oplus$ étant l'addition en arithmétique modulaire. Donc c est une 2-coloration propre de notre G .

Dernièrement, nous voulons démontrer qu'un graphe simple et non-orienté admettant une 2-coloration est bi-parti. Soit $G = (V, E)$ un graphe et $c: V \rightarrow \{0, 1\}$ une 2-coloration propre admise par G . Posons $N = \{v \in V | c(v) = 0\}$ et $B = \{v \in V | c(v) = 1\}$. Alors $V = N \cup B$. CQFD.

Nous avons donc construit formellement un graphe bi-parti représentant l'échiquier mutilé.

Question 2

Nous voulons démontrer $|E| \leq 2|V|$. Nous savons que, $\forall v \in V, \delta(G) \leq \deg(v) \leq \Delta(G) \Rightarrow 2n \leq \sum_{v \in V} \deg(v) \leq 4n, n \in \mathbb{N}$. Or, pour un graphe simple et non-orienté, nous savons aussi que $\sum_{v \in V} \deg(v) = 2|E|$. Donc, $2n \leq \sum_{v \in V} \deg(v) \leq 4n \Rightarrow 2n \leq 2|E| \leq 4n \Rightarrow n \leq |E| \leq 2n$. CQFD.

Question 3

Nous pouvons modéliser les dominos par un couplage : un domino est une arête entre deux sommets distincts (il n'est pas possible de placer un domino sur un autre). Alors si le couplage est parfait, il est possible de paver toutes les cases.

Question 4

```
let construire_gm g m n b =
  let open Ensemble in
  let open Graphe in
  let gm = creer (ordre g) in
  let arc (u,v) = g @?? (u,v) || g @?? (v,u) in
  let ins (u,v) =
    if (u,v) @? m || (v,u) @? m then gm ++ (v,u)
    else gm ++ (u,v)
  in
  (n * b) |? arc |> ins;
  gm
```

Question 5

```
let construire_niveaux gm n b =
  let open Ensemble in
  let open Graphe in
  let h = creer (ordre gm) in
  let niv = Array.make (ordre gm) (-1) in
  let _ = n |> (fun u -> niv.(u) <- 0) in
  let f u =
```

```

    let succ v = Stdlib.(niv.(v) = -1 || niv.(v) = niv.(u) + 1) in
    let visit v = niv.(v) <- Stdlib.(niv.(u) + 1); h ++ (u,v) in
    let _ = voisins gm u |? succ |> visit in ()
in
let _ = bfs gm n f in
let k =
  match (b |? (fun u -> niv.(u) <> -1)) |> (fun u -> niv.(u)) with
  | [] -> -1
  | l -> List.fold_left min max_int l
in h, niv, k

```

Question 6

```

let renverser h =
  let open Graphe in
  let open List in
  let ht = creer (ordre h) in
  let _ = Array.iteri (fun u k -> iter (fun v -> ht ++ (v,u)) k) h in
  ht

```

Question 7

```

let chemins_augmentants h z k =
  let open Graphe in
  let open List in
  let tag = Array.make (ordre h) false in
  let p = ref [] in
  let f u =
    if not tag.(u) then
      let aug = ref false in
      let chemin = ref [] in
      let g v =
        if not !aug && not tag.(v) then begin
          chemin := v :: !chemin;
          if mem v z then aug := true
        end
      in
      dfs h [u] g;
      if !aug then begin
        iter (fun v -> tag.(v) <- true) !chemin;
        p := rev !chemin :: !p
      end
    end
  in
  p

```

```
iter f k;  
!p
```

Question 8

```
let hopcroft_karp g n b =  
  let open Ensemble in  
  let libres e m =  
    let tt x = not (existe m (fun (u, v) -> u = x || v = x)) in  
    e |? tt  
  in  
  let niv_k niv k b = b |? (fun u -> niv.(u) = k) in  
  let rec boucle m =  
    let gm = construire_gm g m n b in  
    let ln = libres n m in  
    let lb = libres b m in  
    let h, niv, k = construire_niveaux gm ln lb in  
    let ht = renverser h in  
    let lz = niv_k niv 0 ln in  
    let lk = niv_k niv k lb in  
    let p = chemins_augmentants ht lz lk in  
    match p with  
  | vide -> m  
  | s -> let c = List.fold_left (+) vide s boucle ((m + c) - (m & c))  
    in  
    boucle vide
```

Implantation

L'intégralité du code du projet est situé dans un fichier OCaml intitulé `hk.ml`. Il suffit de le lancer le programme en lui passant le chemin vers le fichier d'entrée.

Nous avons choisi OCaml comme langage d'implantation pour sa flexibilité lors de l'exploration d'algorithmes plus ou moins complexes. N'ayant aucune contrainte réelle de performance, nous ne nous sommes pas restreint à utiliser un langage comme C ou C++ où nous aurions dû nous occuper de détails relatifs à la machine plutôt qu'au problème présenté.

Conclusion

Nous avons présenté nos réponses aux questions du sujet puis indiqué le fichier contenu le code source, avant de terminer sur une explication du choix du langage OCaml comme implantation.